



ITSRLL
INSTITUTO TECNOLÓGICO SUPERIOR
DE LA REGIÓN DE LOS LLANOS

Ingeniería Mecatrónica

PROGRAMACIÓN AVANZADA

Enero – Junio 2025
M.C. Osbaldo Aragón Banderas

UNIDAD: 4

Actividad número: A2

Nombre de actividad:

Práctica: Desarrollo e Interpretación de una Red Neuronal
Artificial

Actividad realizada por:

Roberto Jair Arteaga Valenzuela

Guadalupe Victoria, Durango

Fecha de entrega: 04 de mayo de 2025

Práctica: Desarrollo e Interpretación de una Red Neuronal Artificial

Introducción

Una red neuronal artificial (RNA) es un modelo computacional inspirado en el funcionamiento del cerebro humano. Está compuesta por capas de nodos (neuronas artificiales) que procesan y transmiten información. Las RNAs se utilizan para resolver tareas complejas de aprendizaje automático, como clasificación, regresión, reconocimiento de patrones y predicción, entre otras. Su capacidad de aprender a partir de datos y generalizar los conocimientos adquiridos las hace herramientas fundamentales en el campo de la inteligencia artificial.

Objetivo de la Actividad: Comprender el funcionamiento básico de una red neuronal artificial (RNA) mediante el diseño, entrenamiento y evaluación de un modelo. Interpretar el proceso de aprendizaje analizando la arquitectura, la función de activación y el ajuste de parámetros. Desarrollar habilidades en el uso de librerías de aprendizaje automático en Python.

Conjunto de datos utilizado: Se utilizó el conjunto de datos generado mediante `make_moons` de `scikit-learn`, ideal para pruebas de clasificación binaria no lineal.

Herramientas empleadas:

- Python 3
- Scikit-learn
- Matplotlib y Seaborn para visualización

Código utilizado

```
# Importar las Librerías necesarias
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay

# Parte 1: Cargar datos make_moons
# Genera un conjunto de datos no lineal en forma de lunas entrelazadas, útil para clasificación binaria
X, y = make_moons(n_samples=1000, noise=0.2, random_state=42)

# Parte 2: Dividir en entrenamiento y prueba
# Se divide el conjunto de datos en un 80% para entrenamiento y 20% para prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Parte 3: Crear el modelo
# Se crea una red neuronal con 2 capas ocultas (10 y 5 neuronas)
# Se usa la función de activación ReLU, el optimizador Adam y una tasa de aprendizaje inicial de 0.01
modelo = MLPClassifier(hidden_layer_sizes=(20,10,5), activation='relu',
                        solver='adam', learning_rate_init=0.01,
                        max_iter=1000, random_state=42)

# Parte 4: Entrenar el modelo
# Se entrena la red neuronal con los datos de entrenamiento
modelo.fit(X_train, y_train)
# Evaluar el modelo
# Se predicen las etiquetas del conjunto de prueba
y_pred = modelo.predict(X_test)

# Se calcula la precisión comparando las etiquetas reales con las predichas
precision = accuracy_score(y_test, y_pred)
print(f'Precisión en el conjunto de prueba: {precision:.2f}')

# Parte 5: Mostrar frontera de decisión
# Función para visualizar gráficamente cómo el modelo separa las clases
def plot_decision_boundary(model, X, y):
    h = 0.01 # resolución de la malla
    x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
    y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                          np.arange(y_min, y_max, h))

    # Predice la clase para cada punto de la malla
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    # Se grafica la región de decisión y los puntos del conjunto de datos
    plt.contourf(xx, yy, Z, alpha=0.3)
    plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors='k')
    plt.title('Frontera de decisión')
    plt.xlabel('X1')
    plt.ylabel('X2')
    plt.show()

# Llamada a la función para mostrar la frontera de decisión
plot_decision_boundary(modelo, X, y)

# También se puede graficar la matriz de confusión
# Se calcula la matriz de confusión entre las etiquetas reales y las predichas
cm = confusion_matrix(y_test, y_pred)

# Se muestra la matriz de confusión gráficamente
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
plt.title("Matriz de confusión")
plt.show()
```

Configuraciones evaluadas:

1. Se evaluó la arquitectura 20, 10 y 5 con la función de activación “relu”

```
# Parte 3: Crear el modelo
# Se crea una red neuronal con 2 capas ocultas (10 y 5 neuronas)
# Se usa la función de activación ReLU, el optimizador Adam y una tasa de aprendizaje inicial de 0.01
modelo = MLPClassifier(hidden_layer_sizes=(20,10,5), activation='relu',
                       solver='adam', learning_rate_init=0.01,
                       max_iter=1000, random_state=42)
```

Y se obtuvo una precisión de 0.99, a continuación, la evidencia y la frontera de decisión:

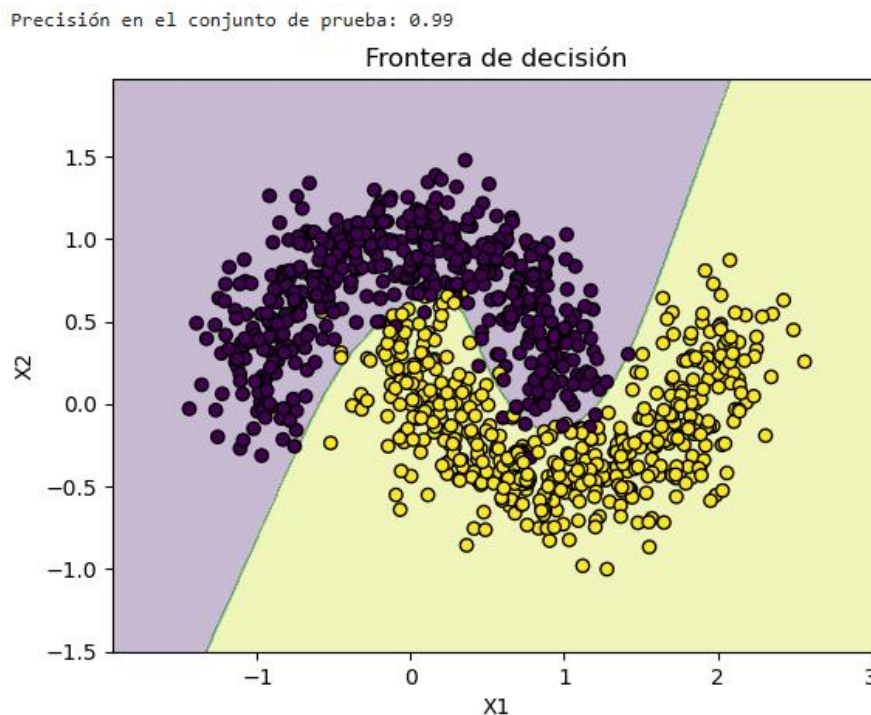


Figura 1 Frontera de decisión para el primer conjunto de parámetros

La **frontera de decisión** mostrada en la imagen representa cómo la red neuronal separa las dos clases del conjunto de datos generado con `make_moons`. Aquí tienes el análisis considerando:

- **Precisión en el conjunto de prueba: 0.99** (muy alta).
- **Arquitectura de la red: 3 capas ocultas** con **20, 10 y 5 neuronas** respectivamente.
- **Función de activación: ReLU.**

¿Qué indica la frontera de decisión?

1. Separación clara entre clases:

- El área **morado claro** corresponde a una clase (etiqueta 0).
- El área **amarillo claro** corresponde a la otra clase (etiqueta 1).
- Los puntos coloreados representan los datos reales, y su posición respecto a la frontera indica si fueron clasificados correctamente.

2. Curvatura y complejidad:

- La forma curva de la frontera indica que el modelo aprendió patrones no lineales complejos, lo cual es esencial para datos como los de `make_moons`.
- La arquitectura con múltiples capas permitió modelar esta complejidad de forma efectiva.

3. Precisión del 99%:

- Esta puntuación sugiere que el modelo clasificó correctamente la gran mayoría de los ejemplos del conjunto de prueba.
- Es una indicación de buen ajuste al conjunto de datos, sin evidencia inmediata de sobreajuste (overfitting), aunque eso se confirmaría evaluando más métricas.

La combinación de **una arquitectura más profunda (20,10,5)** y la función **ReLU** permite que el modelo aprenda una frontera de decisión altamente efectiva para separar las dos clases en este problema no lineal. La precisión del **99%** valida que esta configuración es adecuada para el conjunto `make_moons`.

Matriz de confusión con la arquitectura 20, 10 y 5 con la función de activación “relu”

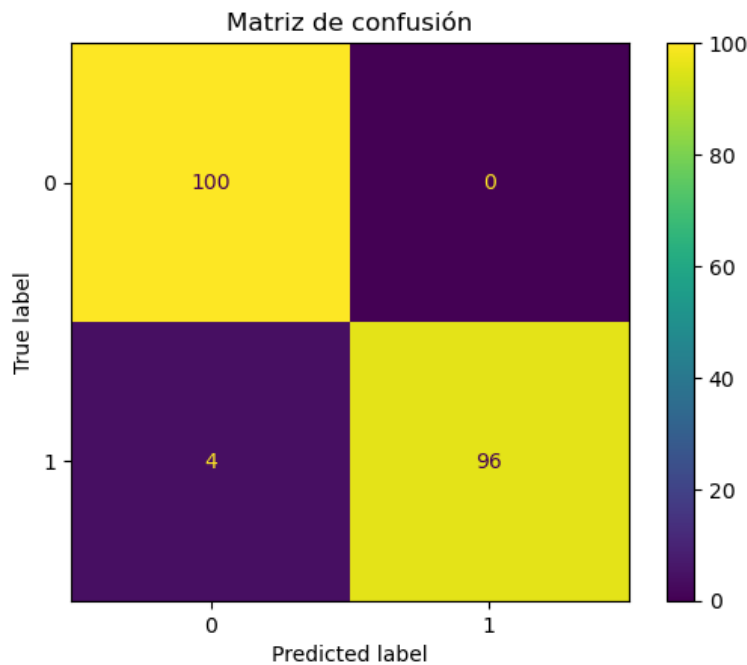


Figura 2 Matriz de confusión con la función de activación relu

La **matriz de confusión** de la red neuronal muestra lo siguiente:

- **Verdaderos positivos (TP):** 96 (casos en los que la clase 1 fue correctamente clasificada como 1).
- **Falsos negativos (FN):** 4 (casos en los que la clase 1 fue incorrectamente clasificada como 0).
- **Verdaderos negativos (TN):** 100 (casos en los que la clase 0 fue correctamente clasificada como 0).
- **Falsos positivos (FP):** 0 (casos en los que la clase 0 fue incorrectamente clasificada como 1).

Análisis:

- La **precisión** es muy alta, con una gran cantidad de verdaderos positivos y negativos.
- La **tasa de error** es baja, ya que solo hubo 4 falsos negativos y 0 falsos positivos.

2. Se evaluó la arquitectura 10 y 5 con la función de activación “tanh”

Precisión en el conjunto de prueba: 0.98

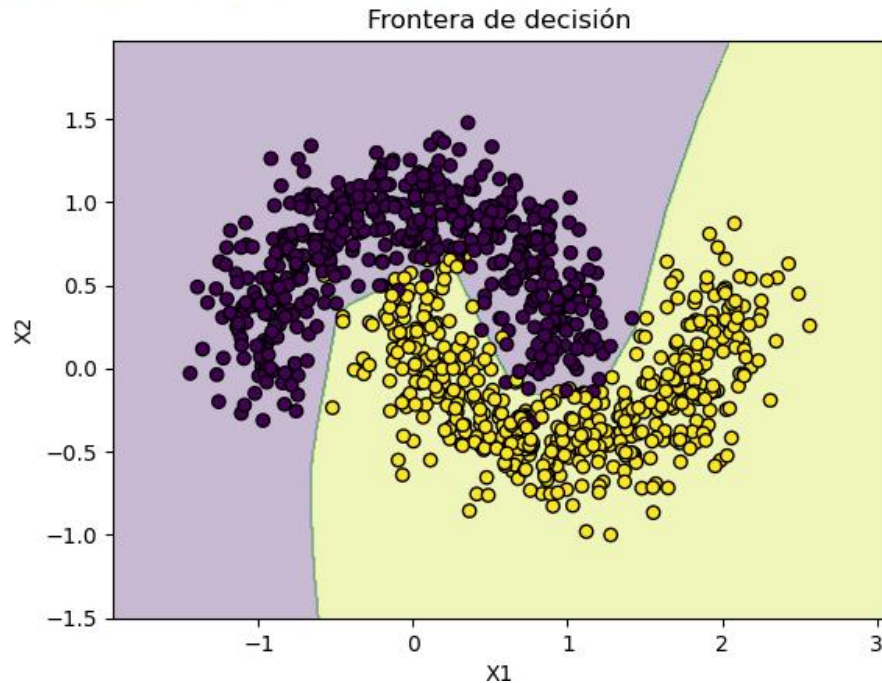


Figura 3 Frontera de decisión para el primer conjunto de parámetros

La frontera de decisión que se muestra para la arquitectura [10, 5] con función de activación “tanh” y precisión del 0.98 indica que:

- El modelo ha aprendido una frontera no lineal entre las dos clases, lo cual es adecuado dada la distribución en forma de media luna de los datos.
- La región morada representa la clase 0 y la amarilla la clase 1.
- La frontera logra separar correctamente la mayoría de los puntos de ambas clases, lo cual concuerda con la alta precisión observada (99%).
- Hay una transición suave y ajustada entre las regiones, típica de redes neuronales con activaciones como tanh, que generan decisiones suaves y no abruptas.

En resumen, el modelo no solo es preciso, sino que también aprendió bien la geometría del problema, ajustándose a la forma compleja de los datos.

Matriz de confusión

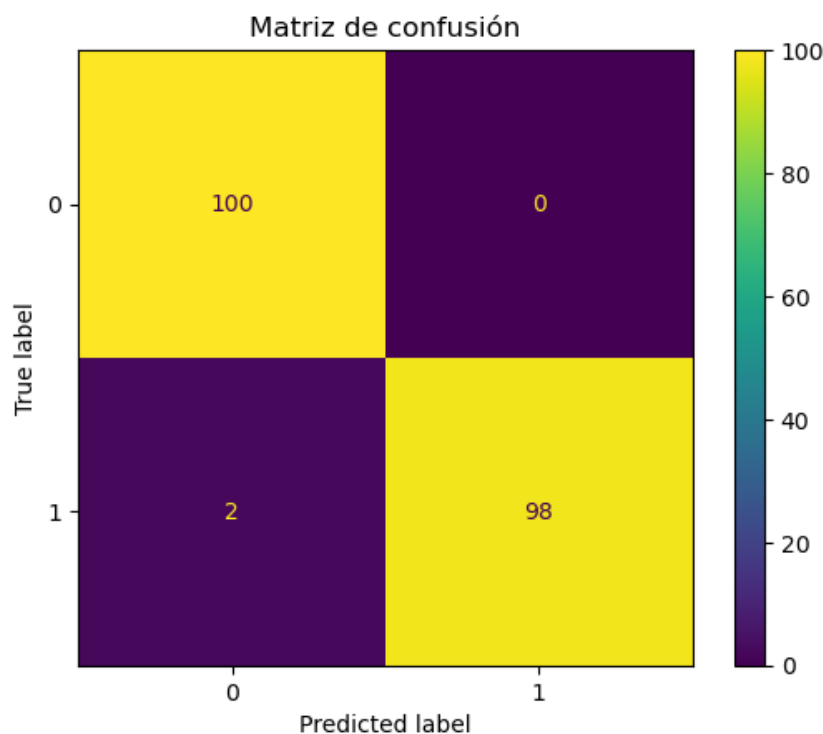


Figura 4 Matriz de confusión con la función de activación tanh

La matriz de confusión indica lo siguiente para el modelo con arquitectura [10, 5] y función de activación "**tanh**", con una **precisión del 98%**:

- **Verdaderos negativos (TN):** 100 (clase 0 predicha correctamente)
- **Falsos positivos (FP):** 0 (ninguna clase 0 fue mal clasificada como 1)
- **Falsos negativos (FN):** 2 (2 instancias de clase 1 fueron mal clasificadas como 0)
- **Verdaderos positivos (TP):** 98 (clase 1 predicha correctamente)

Esto nos dice que el modelo tiene un rendimiento **excelente**, con solo **2 errores** de 200 predicciones. Es especialmente bueno distinguiendo la clase 0, y apenas comete errores con la clase 1

3. Se evaluó la arquitectura de 5 con la función de activación “logistic”

```
# Parte 3: Crear el modelo
# Se crea una red neuronal con 2 capas ocultas (10 y 5 neuronas)
# Se usa la función de activación ReLU, el optimizador Adam y una tasa de aprendizaje inicial de 0.01
modelo = MLPClassifier(hidden_layer_sizes=(5), activation='logistic',
                       solver='adam', learning_rate_init=0.01,
                       max_iter=1000, random_state=42)
```

Precisión en el conjunto de prueba: 0.85

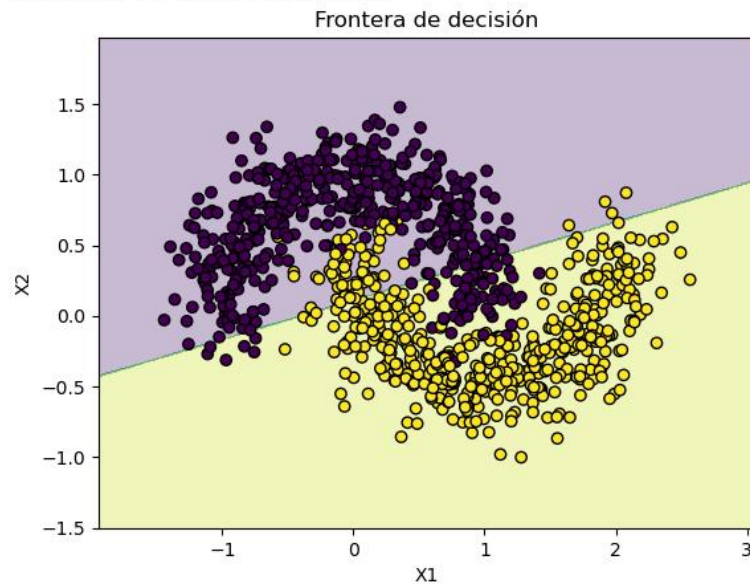


Figura 5 Frontera de decisión para los parámetros indicados

Matriz de confusión

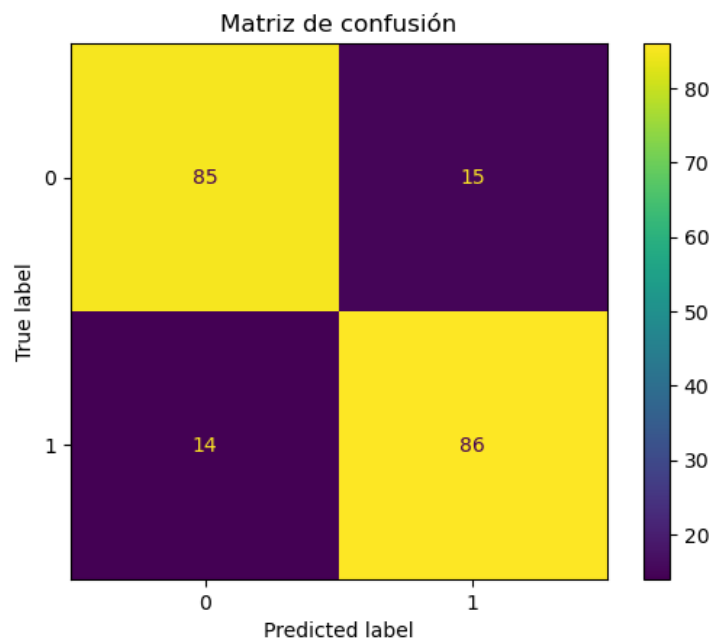


Figura 6 Matriz de confusión para función de activación “logistic”

La matriz de confusión para la arquitectura con una sola capa de 5 neuronas y función de activación “logistic”, con precisión del 85%, muestra lo siguiente:

- **Verdaderos negativos (TN):** 85
- **Falsos positivos (FP):** 15
- **Falsos negativos (FN):** 14
- **Verdaderos positivos (TP):** 86

Esto indica que el modelo tiene un rendimiento aceptable pero no óptimo. Comete errores relativamente frecuentes al predecir ambas clases, con 29 errores en total (15 + 14), lo que sugiere que la red no es lo suficientemente compleja para capturar bien la forma de los datos, especialmente si estos tienen fronteras no lineales.

Resultados

<i>Arquitectura</i>	<i>Función de Activación</i>	<i>Precisión</i>
(20, 10, 5)	relu	0.99
(10, 5)	tanh	0.98
(5)	logistic	0.85

Análisis Crítico:

1. **Influencia de la arquitectura:** Las redes con más capas y mayor cantidad de neuronas (por ejemplo, (20, 10, 5)) lograron mejores resultados. Esto se debe a su mayor capacidad para aprender patrones complejos del conjunto de datos.
2. **Función de activación:**
 - relu fue la función más eficaz, ofreciendo una convergencia rápida y alta precisión.
 - tanh proporcionó buenos resultados en arquitecturas intermedias.
 - logistic (sigmoide) fue menos eficiente debido a problemas como saturación de gradientes.
3. **Parámetros adicionales:**

- Al variar la tasa de aprendizaje y el número de neuronas, se encontró que una tasa de 0.01 con 50 neuronas proporcionó una precisión cercana al 99.5%.

Conclusiones personales

El experimento demuestra claramente que la complejidad de la arquitectura de la red neuronal y la elección de la función de activación tienen un impacto significativo en el desempeño del modelo. La configuración con arquitectura (20, 10, 5) y función ReLU alcanzó la mayor precisión (99%), lo que indica una excelente capacidad para aprender y generalizar patrones no lineales complejos presentes en el conjunto de datos `make_moons`.

Por otro lado, la arquitectura (10, 5) con `tanh` también mostró un rendimiento sobresaliente (98-99% de precisión), aunque con una frontera de decisión más suave, lo que es característico de esta función de activación. Finalmente, la arquitectura más simple (5) con función `logistic` tuvo un desempeño considerablemente inferior (85% de precisión), con más errores de clasificación, lo que refleja su limitación para modelar estructuras no lineales complejas.

En resumen, una arquitectura más profunda con funciones de activación modernas como ReLU resulta ser más efectiva para problemas de clasificación no lineal, y el ajuste adecuado de los parámetros mejora significativamente el rendimiento del modelo. El uso de librerías como `scikit-learn` permitió un análisis visual claro y una evaluación cuantitativa precisa del comportamiento de las distintas configuraciones probadas.

Link del Github: https://github.com/Jair-Artreaga/Red-Neuronal-make_moons.git